Documento de Diseño

Anglo Electric

6.1 Arquitectura

Componentes principales y responsabilidades:

- **Frontend (Cliente):** Aplicación web desarrollada en React. Es responsable de renderizar la interfaz de usuario, gestionar el estado del carrito de compras y consumir los servicios de la API.
- **Backend (API RESTful):** Servidor desarrollado en Python con FastAPI. Centraliza la lógica de negocio, procesa la autenticación, gestiona el CRUD del catálogo y expone los endpoints para el cliente.
- **Base de Datos (Neon):** Motor relacional PostgreSQL alojado en la nube de Neon. Almacena de forma persistente y segura los datos de usuarios, roles, productos y cotizaciones.
- **Módulo de Inteligencia Artificial:** Script integrado en el backend que utiliza Scikit-Learn (KNN) para procesar los datos de entrada (consumo en kWh y presupuesto) y retornar el ID del kit fotovoltaico óptimo.

Comunicación:

El Frontend se comunica con el Backend a través del protocolo HTTPS utilizando peticiones REST (GET, POST, PUT, DELETE). La API devuelve las respuestas en formato JSON. El Backend se conecta a la base de datos PostgreSQL en Neon mediante la librería SQLAlchemy utilizando una URI de conexión segura.

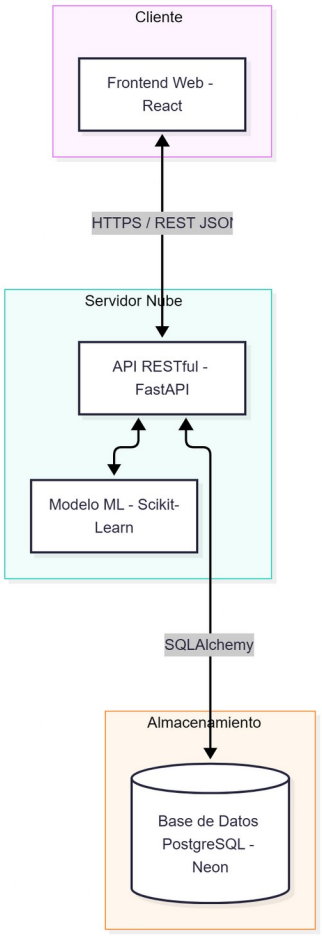


Figura 1: Diagrama de bloques de la arquitectura (React → HTTPS → FastAPI → PostgreSQL Neon)

6.2 Modelo de datos

El sistema utiliza un modelo relacional en PostgreSQL. Las entidades principales incluyen:

- **roles:** Define los permisos (id, nombre).
- **usuarios:** Almacena credenciales (id, nombre, email, password_hash, rol_id). Clave foránea a roles. Restricción UNIQUE en email.
- **categorias:** Clasifica el inventario (id, nombre).
- **productos:** Detalles del equipo (id, nombre, precio, stock, categoria_id, potencia_w). Clave foránea a categorias.
- **cotizaciones:** Cabecera de la solicitud comercial (id, usuario_id, estado, fecha). Clave foránea a usuarios.
- **detalle_cotizaciones:** Relación muchos a muchos entre cotizaciones y productos (cotizacion_id, producto_id, cantidad, precio_unitario).

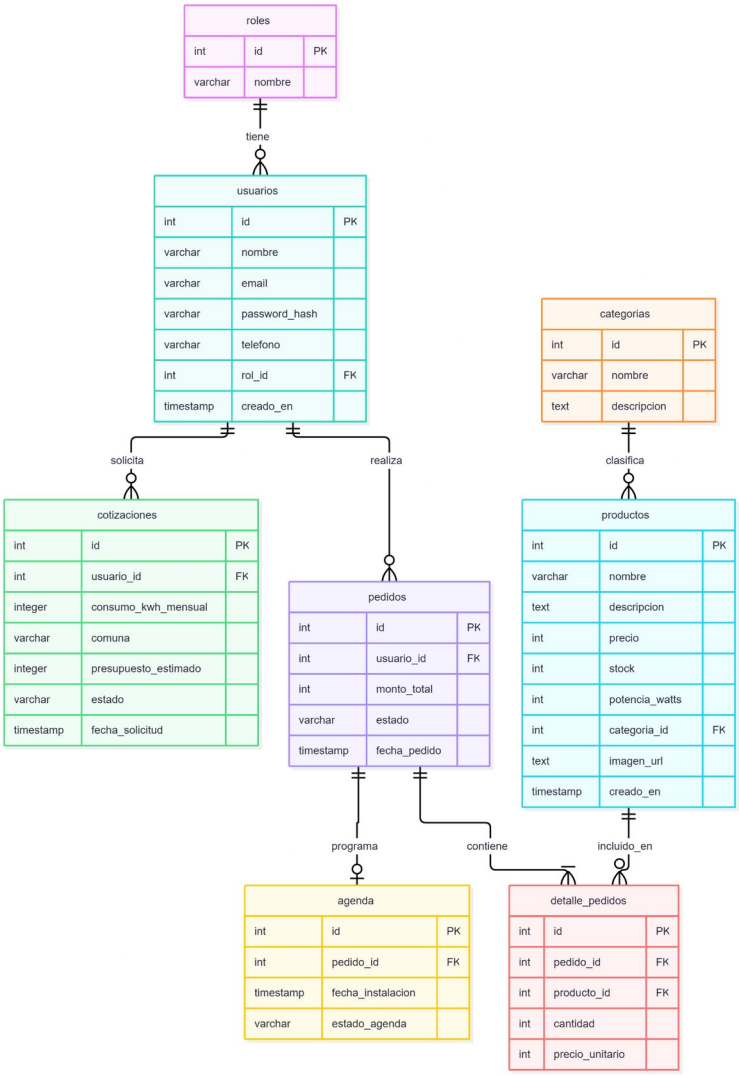


Figura 2: Modelo entidad-relación de la base de datos

6.3 UML mínimo

- **Diagrama de Casos de Uso:** Muestra al "Cliente" interactuando con "Registrarse", "Ver Catálogo" y "Cotizar Kit con IA". Muestra al "Administrador" interactuando con "Gestionar Productos".
- **Diagrama de Clases:** Muestra las clases del backend (Usuario, Producto, Cotización) y sus atributos principales.

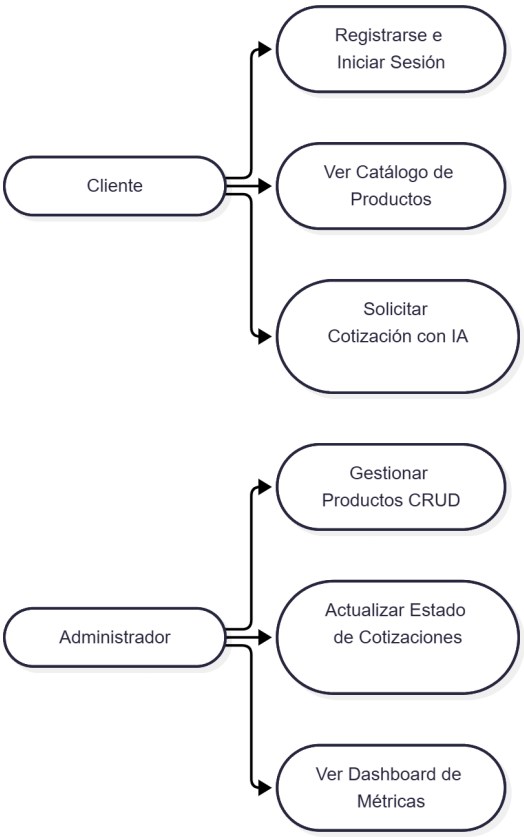


Figura 3: Diagrama de casos de uso

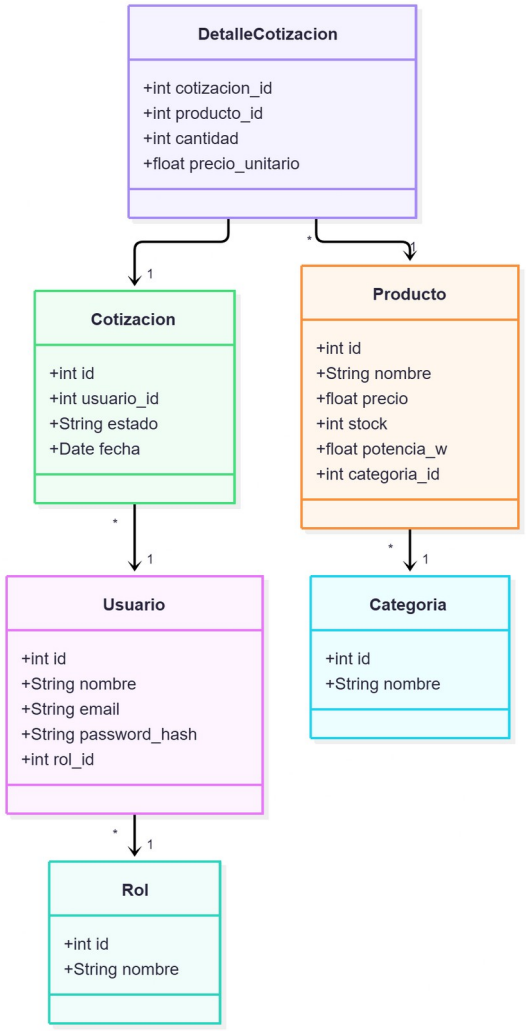


Figura 4: Diagrama de clases

- **Diagrama de Secuencia (Cotización con IA):**
  1. Cliente envía POST /recomendar-kit/ (con kWh y presupuesto).
  2. FastAPI recibe y valida el token JWT.
  3. FastAPI envía los datos al modelo KNN.
  4. KNN retorna el ID del kit.
  5. FastAPI consulta la DB (Neon) por los detalles del kit.
  6. FastAPI devuelve el JSON al Cliente.

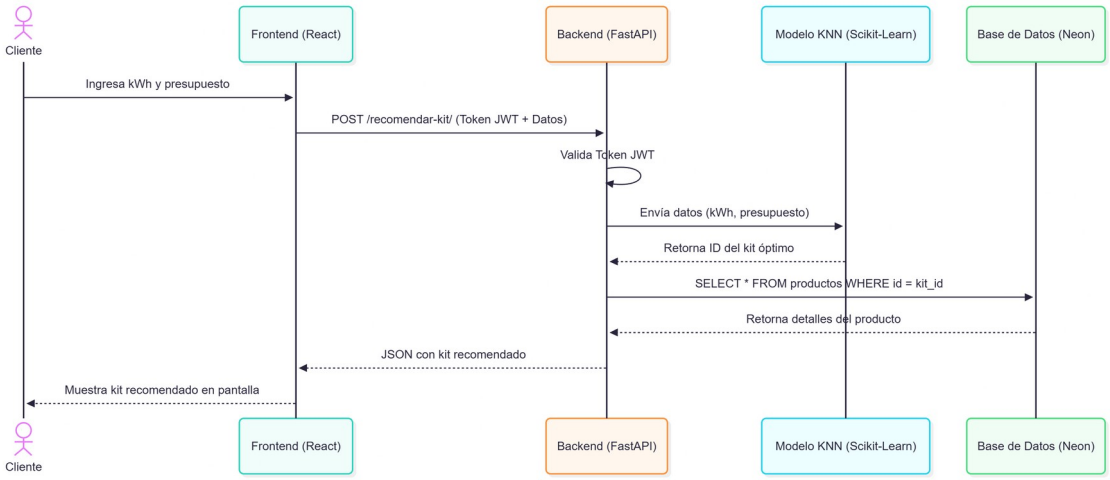


Figura 5: Diagrama de secuencia (Cotización con IA)

- **Diagrama de Componentes:** Frontend (React) conectado a Backend (FastAPI), que a su vez se conecta a DB (Neon) y Modelo ML (Scikit-Learn).

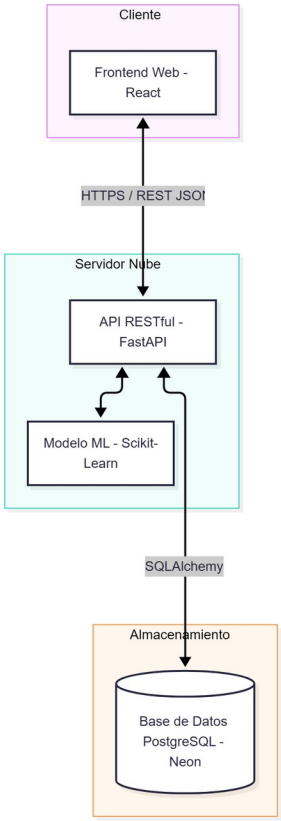


Figura 6: Diagrama de componentes

6.4 Decisiones técnicas

Decisión	Alternativas consideradas	Razón	Consecuencia
Backend con FastAPI	Django, Flask	Rapidez de ejecución asíncrona, validación automática de datos con Pydantic y auto-generación de documentación en Swagger.	Requiere manejo cuidadoso de sesiones asíncronas con SQLAlchemy.
Base de datos PostgreSQL en Neon	MySQL local, MongoDB	Modelo serverless que permite despliegue rápido en la nube sin configurar servidores. Integridad relacional estricta para compras.	Se requiere conexión constante a internet para pruebas de desarrollo local.
Seguridad con JWT y Bcrypt	Sesiones basadas en cookies (Session IDs)	Arquitectura stateless (sin estado) que no satura la memoria del servidor. Ideal para consumir desde React.	El frontend (React) debe encargarse de almacenar el token de forma segura y gestionar su expiración.
IA con algoritmo KNN	Redes Neuronales, Árboles de Decisión	Es un modelo de clasificación basado en similitud que se ajusta perfectamente a la recomendación de kits fotovoltaicos pre-armados según historial de consumo cercano.	Requiere un preprocesamiento cuidadoso y normalización de los datos de entrada por parte del ingeniero de datos.

6.5 Requisitos no funcionales en el diseño

- **Seguridad:** Atendido mediante la decisión de almacenar contraseñas exclusivamente mediante hashes irreversibles (Bcrypt) y proteger todos los endpoints de modificación de datos con tokens JWT de duración limitada.
- **Rendimiento:** Atendido mediante el uso del framework FastAPI, uno de los más veloces en Python gracias a su arquitectura asíncrona (ASGI), capaz de soportar múltiples peticiones concurrentes de los clientes.
- **Escalabilidad:** Atendido al separar el frontend del backend (arquitectura de microservicios lógicos) y utilizar una base de datos en la nube (Neon) que escala automáticamente según la demanda computacional.
- **Portabilidad:** Atendido mediante el uso de contenedores Docker, garantizando que el sistema funcione idénticamente en cualquier entorno de desarrollo o producción sin conflictos de dependencias.

6.6 Diseño Docker

Para garantizar la portabilidad, el proyecto se empaqueta en contenedores independientes orquestados por Docker Compose:

- **Servicio backend:** Construido con un Dockerfile basado en python:3.10-slim. Expone el puerto 8000. Ejecuta el servidor Uvicorn con FastAPI y contiene las librerías de IA (Scikit-Learn).

- **Servicio frontend:** Construido con un Dockerfile basado en node:18-alpine. Expone el puerto 3000. Ejecuta la interfaz de React.
- **Base de datos:** Al estar alojada de forma nativa en la nube (Neon serverless), no requiere un contenedor Docker local de PostgreSQL, aliviando la carga de memoria en la máquina anfitriona.
- **Variables de entorno:** La conexión a la base de datos y la clave secreta de JWT se inyectan a los contenedores mediante un archivo .env privado. El repositorio solo almacena un archivo .env.example con la estructura base (DATABASE_URL=, SECRET_KEY=).